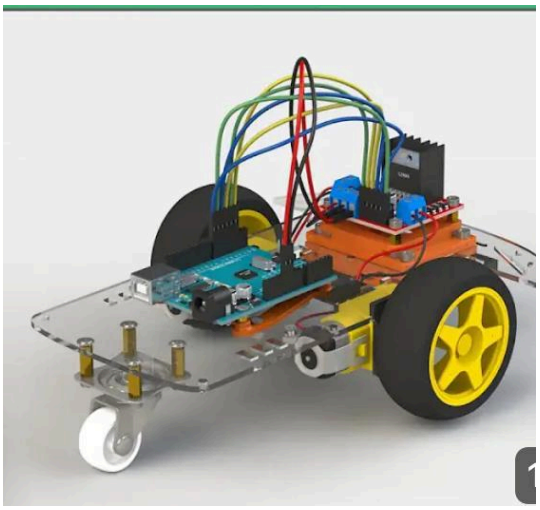
PH102 PROJECT

VOICE CONTROLLED ARDUINO CAR



Group 7

1)Isha B Nambiar-2024MEB1351

2)Ishan Gangwani-2024MEB1352

3)Jaskaran Kora-2024MEB1353

4)Jasmeet Singh Chopra-2024MEB1354

5)Kaashika Chopra-2024MEB1355

6)Kanav Dumra-2024MEB1356

Introduction:

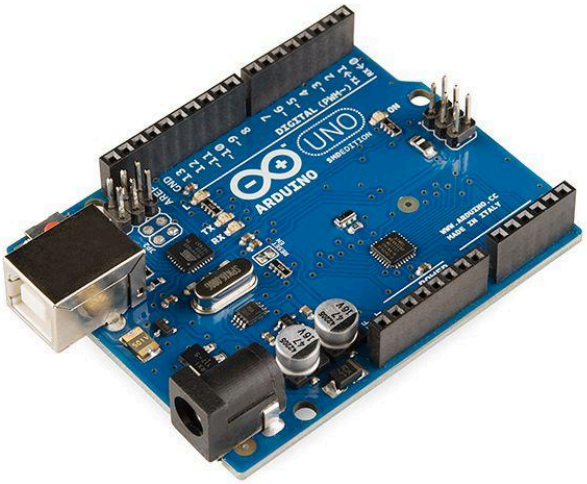
This project aims to build a voice-controlled bot using Arduino. It combines robotics, voice recognition, and microcontroller programming to create a bot that responds to voice commands, allowing for hands-free and intuitive interaction. With recent advancements in voice recognition technology, voice-controlled devices are becoming increasingly popular in applications ranging from home automation to assistive robotics. Our project aims to bring this innovation to a simple robotic platform, demonstrating how an Arduino can interpret voice commands and control a bot's movements.

This project showcases the integration of hardware and software to create an interactive robot capable of responding to human speech. The goal is to demonstrate how voice commands can enhance user interaction with technology, making robotics more accessible and user-friendly. In this report, we will discuss the components used, the system architecture, the code implementation and the challenges faced.

Parts:

- Arduino UNO
- Ultrasonic Sensor
- Bluetooth module
- Motor Driver L293D
- Servo motor
- Gear motor * 2
- Robot Wheel * 2
- Castor Wheel * 1
- Battery holder
- Li ion battery *2
- Jumper wires
- Cardboard

ARDUINO UNO

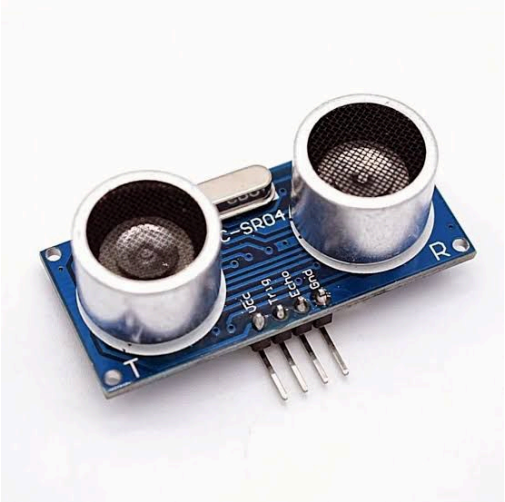


Arduino is an open-source electronics platform based on easy-to-use hardware and software. It consists of a programmable microcontroller board and a user-friendly programming environment. Designed to be flexible, Arduino enables users to create a wide range of interactive projects, from simple LED displays to complex robotics systems.

Some important parts of an Arduino Uno board are:

- ATmega328 Microcontroller
- Digital I/O Pins
- Analog Input Pins
- Power Pins
- USB Port
- TX and RX LEDs

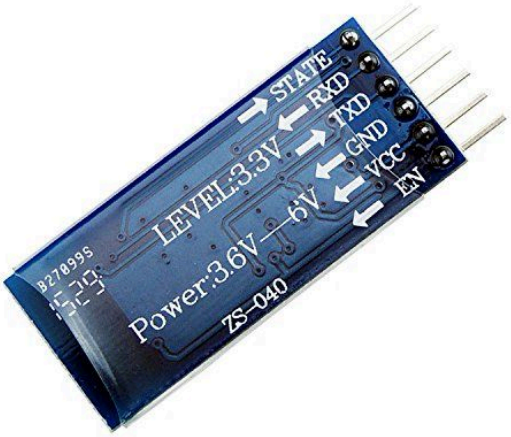
ULTRASONIC SENSOR



An ultrasonic sensor is a device that uses sound waves to measure the distance to an object. It is commonly used in robotics, automotive applications, and industrial automation due to its accuracy, simplicity, and affordability. Ultrasonic sensors are especially useful for detecting objects, measuring distances, and avoiding obstacles in environments where other types of sensors (e.g., infrared) might be less effective due to lighting or color interference. A typical ultrasonic sensor, like the popular HC-SR04, has the following key components:

- Transmitter (Ultrasonic Emitter): The transmitter generates high-frequency sound waves, usually in the range of 40 kHz. It sends out a burst of sound pulses when triggered.
- Receiver (Ultrasonic Receiver): The receiver listens for the sound waves reflected back from the object.
- Control Circuit: The control circuit manages the timing, triggering, and signal processing of the transmitted and received sound waves.
- Echo and Trigger Pins: The sensor has a trigger pin to initiate the ultrasonic pulse and an echo pin to read the received pulse after it bounces back.

BLUETOOTH MODULE



A **Bluetooth module** is a device that allows for wireless communication between different devices using Bluetooth technology. Bluetooth is a wireless communication standard designed for short-range (typically up to 100 meters) data exchange between devices. The **HC-05** is one of the most commonly used Bluetooth modules in Arduino and robotics projects.

SERVO MOTORS

Unlike regular DC motors, which run continuously when powered, servo motors can rotate to a specific position, hold that position, and even move between two points with accuracy. It consists of a small electric motor (usually a **DC motor**) paired with a **feedback mechanism** (usually an encoder or potentiometer) to

provide precise control of its position. The motor is controlled by a **servo controller**, which sends a signal (usually PWM) to determine the motor's desired position.

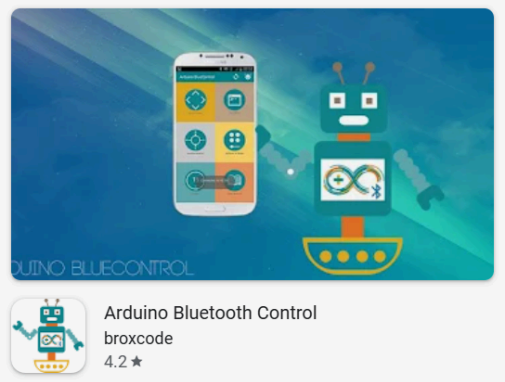
Procedure:

1. For the chassis, cut a rectangle out of the cardboard. Make 2 holes in the middle so that the jumper wires can pass through between the motor and the arduino.
2. Place the motors at the two corners of the cardboard, hot glue them to the base.
3. Connect the arduino Uno with the motor driver. Hot glue them to the opposite side of the cardboard base.
4. Connect the jumper wires from the motors to the motor driver.
5. Hot glue the servo motor to the front of the bot. Connect it to the motor driver using wires.
6. Glue the ultrasonic sensor to the servo motor. This will allow us to detect obstacles in 180^0 by rotating the servo.
7. Connect the ultrasonic sensor to the motor driver by making the following connections from sensor to motor driver:
    a. GND—----> GND
    b. Echo—-----> AO
    c. Trig—-----> A1
    d. VCC—---> 5v
8. Connect the bluetooth module to the motor driver by making the following connections:
    a. RXD—------>TX(D1)
    b. TXD—------->RX(D0)
    c. GND—------>GND
    d. VCC—------>5v
9. Attach the Bluetooth module, Battery holder, castor wheel, Switch onto the bot.
10. Remove the RX and TX jumper wires. Connect the arduino to the computer and upload the code. Reconnect the jumper wires.
11. Attach wheels and battery

Arduino bluetooth control

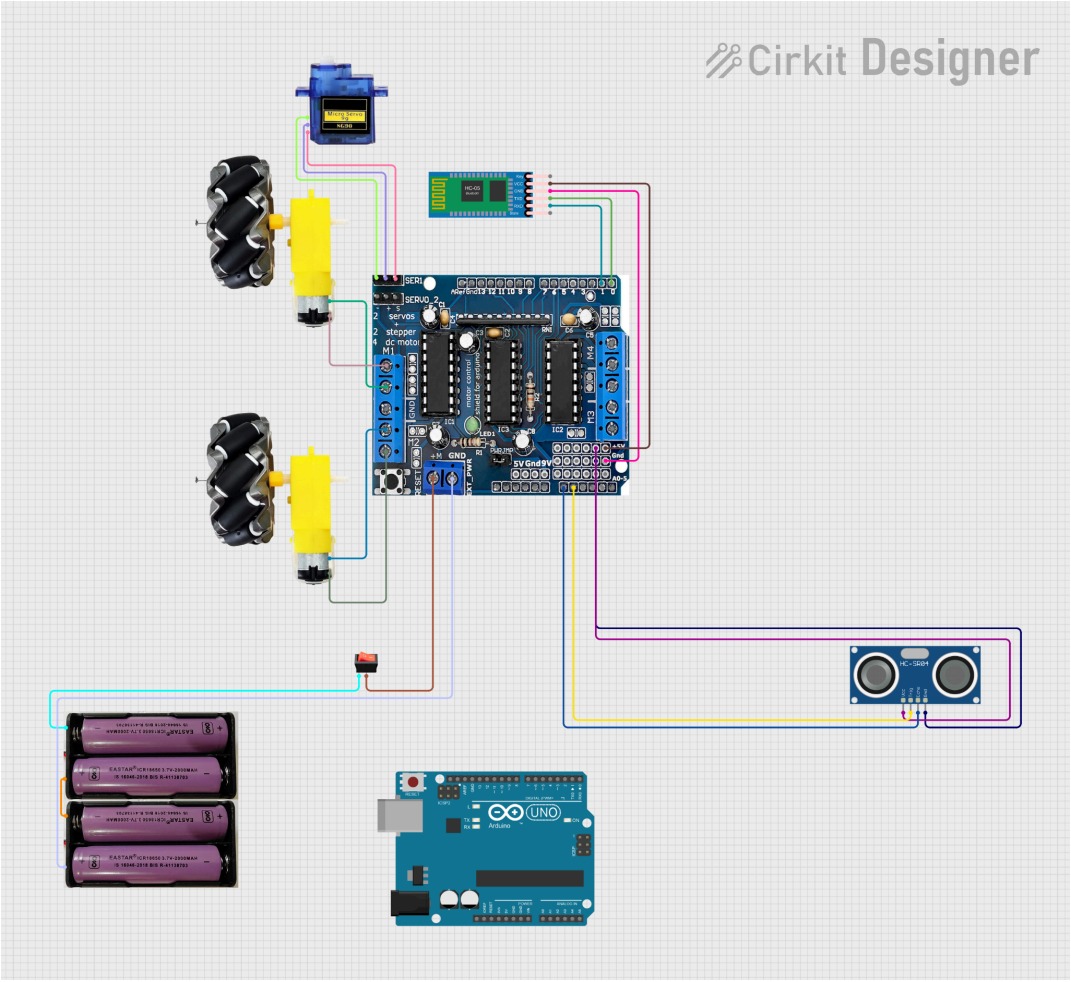
Bluetooth Control:

1. Install the Arduino Bluetooth Control App on your phone.



2. Connect the app to the module. Enter the commands and the data which the arduino should receive along with it.

Circuit:



CODE

Arduino code (also called Arduino sketch) includes two main parts: setup code and loop code.

Setup Code

- Is code in setup() function.
- Executed right after power-up or reset
- Executed only one time.
- Used to initialize variables, pin modes, start using libraries,

Loop Code

- Is code in loop() function.
- Executed right after setup code.
- Executed repeatedly (infinitely).
- Used to do the main task of application

code_main_wala.ino

```
/*obstacle avoiding, Bluetooth control, voice control robot car.
    Home Page
*/
#include <Servo.h>
#include <AFMotor.h>
#define Echo A0
#define Trig A1
#define motor 10
#define Speed 100
#define spoint 103
char value;
int distance;
int Left;
int Right;
int L = 0;
int R = 0;
int L1 = 0;
int R1 = 0;
Servo servo;
AF_DCMotor M1(1);
AF_DCMotor M2(2);
AF_DCMotor M3(3);

AF_DCMotor M4(4);
void setup() {
  Serial.begin(9600);
  pinMode(Trig, OUTPUT);
  pinMode(Echo, INPUT);
  servo.attach(motor);
  M1.setSpeed(Speed);
  M2.setSpeed(Speed);

}
```

code_main_wala.ino

```
void loop() {

  //Obstacle();
  Bluetoothcontrol();
  voicecontrol();
}
void Bluetoothcontrol() {
  if (Serial.available() > 0) {
    value = Serial.read();
    Serial.println(value);
  }
  if (value == 'F') {
    forward();
  } else if (value == 'B') {
    backward();
  } else if (value == 'L') {
    left();
  } else if (value == 'R') {
    right();
  } else if (value == 'S') {
    Stop();
  }
}
void Obstacle() {
  distance = ultrasonic();
  if (distance <= 12) {
    Stop();
    backward();
    delay(100);
    Stop();
    L = leftsee();
    servo.write(spoint);
    delay(800);
```

code_main_wala.ino

```
    Stop();
    L = leftsee();
    servo.write(spoint);
    delay(800);
    R = rightsee();
    servo.write(spoint);
    if (L < R) {
      right();
      delay(500);
      Stop();
      delay(200);
    } else if (L > R) {
      left();
      delay(500);
      Stop();
      delay(200);
    }
  } else {
    forward();
  }
}
void voicecontrol() {
  distance = ultrasonic();
  if (distance <= 12) {
    Stop();


  } else {
    if (Serial.available() > 0) {
    value = Serial.read();
    Serial.println(value);
    if (value == '^') {
      forward();
```

code_main_wala.ino

```
    }
    else if (value == '-') {
      backward();
    }
    else if (value == '<') {
      L = leftsee();
      servo.write(spoint);
      if (L >= 10 ) {
        left();
        delay(500);
        Stop();
      } else if (L < 10) {
        Stop();
      }
    }
    else if (value == '>') {
      R = rightsee();
      servo.write(spoint);
      if (R >= 10 ) {
        right();
        delay(500);
        Stop();
      } else if (R < 10) {
        Stop();
      }
    } else if (value == '*') {
      Stop();
    }

  }


}
```

code_main_wala.ino

```
}
// Ultrasonic sensor distance reading function
int ultrasonic() {
  digitalWrite(Trig, LOW);
  delayMicroseconds(4);
  digitalWrite(Trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(Trig, LOW);
  long t = pulseIn(Echo, HIGH);
  long cm = 0.017 * t; //time convert distance
  return cm;
}
void forward() {
  M1.run(FORWARD);
  M2.run(FORWARD);

}
void backward() {
  M1.run(BACKWARD);
  M2.run(BACKWARD);

}
void right() {
  M1.run(FORWARD);
  M2.run(BACKWARD);

}
void left() {
  M1.run(BACKWARD);
  M2.run(FORWARD);

}
void Stop() {
```

```
void Stop() {
  M1.run(RELEASE);
  M2.run(RELEASE);

}
int rightsee() {
  servo.write(20);
  delay(800);
  Left = ultrasonic();
  return Left;
}
int leftsee() {
  servo.write(180);
  delay(800);
  Right = ultrasonic();
  return Right;
}
```

Working Principle

- The code for the bot is stored in the arduino UNO, which is then connected to the bluetooth module.

The HC-05 Bluetooth Module has 6 pins:

- **Enable/Key pin:** This pin is used to toggle between Data Mode (set **LOW**) and Command mode (set **HIGH**). If not connected, it is in Data mode by default
- **VCC pin:** power pin, connect this pin to +5V of Supply voltage
- **GND pin:** power pin, connect this pin to the **GND** of the power source
- **TX pin:** Serial data pin, connect this pin to the RX pin of Arduino. The data received via Bluetooth will be sent to this pin as serial data.
- **RX pin:** Serial data pin, connect this pin to the TX pin of Arduino. The data received from this pin will be sent to Bluetooth
- **State:** The state pin is connected to the onboard LED, it can be used as feedback to check if Bluetooth is working properly.

- The Arduino connects to bluetooth module via the Serial pins
- The bluetooth module receives data through the Serial RX pins and transmits data to the smartphone via bluetooth
- Receives data via bluetooth and transmits it to arduino via Serial TX pins
- The Arduino Bluetooth Control App is a mobile app that communicates with Arduino via Bluetooth. You can interact with Arduino via this app as if Serial Monitor on your PC, without adding any special code for the Bluetooth module in your Arduino code, by doing the following step:
    - Connect Arduino to HC-05 Bluetooth module
    - Install the app on your smartphone
    - Open the App and pair it with the HC-05 Bluetooth module

Thus we control the bot using the Bluetooth app.

Problems Faced:

- Two pins were faulty in motor driver
- Not enough powerful battery
- Weak soldering

Contributions of each member

- Research on Code: Ishan
- Research on Motors: Isha
- Research on Bluetooth Module: Kaashika
- Research on Arduino: Kanav
- Research on Assembly: Jasmeet
- Research on Sensor: Jaskaran

Conclusion:

In this project, we successfully designed and implemented a voice-controlled Arduino car, utilising a combination of hardware components and software programming. Throughout the project, we encountered and overcame challenges related to speech recognition accuracy, signal processing, and hardware integration. The use of an Android phone or a voice recognition module enabled the conversion of spoken commands into signals that the Arduino could process to control the car's motors. With further improvements in speech recognition accuracy and hardware optimization, such systems could be applied in various domains, such as assistive technologies, security robots, and smart transportation systems.